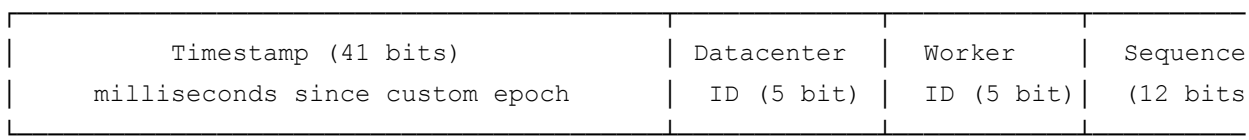
snowflake-id

A high-performance, zero-dependency 64-bit distributed unique ID generator for Node.js, inspired by Twitter's Snowflake algorithm.

Bit Layout



Property	Value
Max throughput	**4,096 IDs/ms** per worker
Unique workers	1,024 (32 datacenters × 32 workers)
Generator lifespan	~69 years from epoch
ID size	64-bit integer (returned as BigInt)

Quick Start

```
const { createGenerator } = require('./index');

const gen = createGenerator({ workerId: 1, datacenterId: 0 });

gen.nextId();          // → 929310234123423744n  (BigInt)
gen.nextIdString();    // → '929310234123423744'  (safe for JSON/DBs)
gen.nextIdHex();       // → '0ce9f4ab3d800001'    (hex, 16 chars)
gen.nextIds(5);        // → [BigInt, BigInt, ...]  (bulk)
```

API

`createGenerator(options?)`

Creates a single `SnowflakeGenerator`.

Option	Type	Default	Description
`workerId`	`number`	`0`	Worker node ID (0–31)
`datacenterId`	`number`	`0`	Datacenter ID (0–31)
`epoch`	`bigint|number`	`1577836800000`	Custom epoch (ms, Unix)

```
const gen = createGenerator({ workerId: 3, datacenterId: 1 });
```

`gen.nextId()` ➜ `BigInt`

Generate one ID. **Always use BigInt** — Snowflake IDs exceed `Number.MAX_SAFE_INTEGER`.

`gen.nextIdString()` ➜ `string`

Generate one ID as a decimal string. Safe for JSON serialization and databases.

`gen.nextIdHex()` ➜ `string`

Generate one ID as a zero-padded 16-character hex string.

`gen.nextIds(count)` ➜ `BigInt[]`

Generate `count` IDs in bulk.

`gen.parse(id)` ➜ `object`

Decode a Snowflake ID back into its components.

```
gen.parse(92931023412342374n);
// → {
//     id:           '929310234123423744',
//     timestamp:    1704067200123,
//     date:         Date { 2024-01-01T00:00:00.123Z },
//     datacenterId: 0,
//     workerId:     1,
//     sequence:     0
//   }
```

`createPool(options?)`

Creates a `SnowflakePool` that round-robins across multiple workers. Use this when you need throughput beyond 4,096 IDs/ms.

```
const { createPool } = require('./index');

const pool = createPool({ datacenterId: 0, workerIds: [0, 1, 2, 3] });
pool.nextId();       // → BigInt (round-robins across workers 0–3)
pool.nextIds(1000);  // → BigInt[]
```

`parse(id, epoch?)` (static)

Parse a Snowflake ID without creating a generator instance.

```
const { parse } = require('./index');
parse('929310234123423744');
```

`SnowflakeGenerator.limits`

```
SnowflakeGenerator.limits;
// → {
//     maxDatacenterId: 31,
//     maxWorkerId:     31,
//     maxSequence:     4095,
//     maxIdsPerMs:     4096,
//     lifespanYears:   69
//   }
```

Notes on BigInt

JavaScript's `number` type loses precision above `2^53 - 1` (~9 quadrillion). Snowflake IDs are 64-bit integers and will exceed this. **Always store and transmit IDs as strings.**

```
// ✗ Dangerous — precision loss
JSON.stringify({ id: Number(gen.nextId()) });
```

```
// ✓ Safe
JSON.stringify({ id: gen.nextIdString() });
```

Running Tests & Benchmarks

```
node test/run.js
```

File Structure

```
snowflake-id/
├── index.js                  # Public API (factory functions + re-exports)
├── src/
│   ├── SnowflakeGenerator.js # Core ID generation logic
│   └── SnowflakePool.js      # Multi-worker pool
└── test/
    └── run.js                # Functional tests + performance benchmark
```